

Analysis Report Template

Student Full Name	Joel Da Costa	Student ID No.	W2154055
Case Study Title			
Case Study (A): Predicting Clients Loan Status.			[Total 43 Marks]
Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?			

Task (1) – Domain Understanding: Classification

The financial risk analysts decided that classification modelling is required. Indicate in the table below for each of the listed variables in your data which ones you should RETAIN and can be included in the classification modelling of loan approval status you should DROP (REMOVE). Justify your decision logically and/or by research (for any research, include in-text citation using the Harvard style) [3 Marks]		
Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	Drop	Identifying variable does not assist in model and could bring noise to the dataset
Sex	Drop	Large data missingness, cannot be used for modelling or even imputation
Age	Retain	Usable demographic variable that can be useful in modelling as well as showing eligibility in real world cases of loan approval
Education Qualifications	Drop	Large informative information that cannot assist the dataset, 'unknown' values don't assist in informative information
Income	Retain	Usable affordability variable that directly assists in the loan approval status
Home Ownership	Retain	Variable is often an indicator of repayment stability, can be used for modelling.
Employment Length	Retain	Huge indicator that determines the stability of loan repayments, very useful for modelling
Loan Intent	Retain	Once encoded this variable will be useful, it also gives relevant context to the applications, which is reasoning toward loan acceptance
Loan Amount	Retain	Relevant affordability indicator and highly important in approval decisions.
Loan Interest Rate	Retain	High relevancy to lending opportunities
Loan-to-Income Ratio (LTI)	Retain	Lenders use this indicator to calculate loan amount (HomeOwners Alliance, 2018)
Payment Default on File	Retain	Great usage for this variable, indicates credit risk
Credit History Length	Retain	Relevant credit profile indicator

Loan Approval Status	Retain	This is the classification target
Maximum Loan Amount	Retain	This is the regression target
Credit Application Acceptance	Drop	This variable is clearly data leakage and cannot be used for modelling
References		
HomeOwners Alliance. (2018). <i>What Percentage Of Income Should Go To Your Mortgage?</i> - HomeOwners Alliance. [online] Available at: https://hoa.org.uk/advice/guides-for-homeowners/i-am-buying/how-much-can-i-afford/ .		

Task (2) – Exploring and Understanding Your Dataset

With the aid of your Final Python Notebook 1, for your RETAINED input variables and your class “Target” variable, produce 1) **basic descriptive stats** and 2) **variable scale type**, then 3) **plot the distribution of your target variable**. (Paste the three screenshots of code OUTPUTS ONLY for evidence of these elements).

[2 Marks]

```
# Returns all the data types for all variables in this dataset after dropping variables
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10919 entries, 0 to 10918
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   10917 non-null  float64
1   income                10888 non-null  float64
2   home_ownership        10918 non-null  object
3   employment_length     10903 non-null  float64
4   loan_intent           10918 non-null  object
5   loan_amount           10918 non-null  float64
6   loan_interest_rate    10916 non-null  float64
7   loan_income_ratio     10918 non-null  float64
8   payment_default_on_file 10916 non-null  object
9   credit_history_length  10918 non-null  float64
10  loan_approval_status   10917 non-null  object
11  max_loan_amount(allowed) 10918 non-null  float64
dtypes: float64(8), object(4)
memory usage: 1023.8+ KB
```

Figure 1 - Data types

```
# Display the descriptive values for the numeric variables in the dataset after dropping variables
data.describe()

count    10917.000000    10888.000000    10903.000000    10918.000000    10916.000000    10918.000000    10918.000000    10918.000000    10918.000000    1.091800e+04
mean      27.120454      44903.095151      3.613868      10109.641878      12.621625      0.235711      5.600476      4.990929e+03
std       6.809568      24315.984295      3.983275      6283.259378      3.743181      0.114349      4.212733      2.537449e+04
min       1.000000       240.000000      0.000000      700.000000      -11.140000      0.010000      2.000000      -2.426900e+06
25%      23.000000      30000.000000      1.000000      5000.000000      10.590000      0.140000      3.000000      0.000000e+00
50%      25.000000      38000.000000      3.000000      9000.000000      12.920000      0.230000      4.000000      0.000000e+00
75%      29.000000      54996.000000      5.000000      14000.000000      14.960000      0.330000      6.000000      0.000000e+00
max      180.000000     850000.000000     35000.000000     150.000000      150.000000      0.830000      30.000000      3.500000e+04
```

Figure 2 - Descriptive stats



Figure 3 - Plot for target variable

Task (3) – Data Preparation: Cleaning and Transforming your data

a) With the aid of your Final Python Notebook 1, when you first explored your retained variables in the loan dataset, you may have found some issues. **1) Report any issues you found in your retained dataset variables.** Based on the issues you found in your data, **2)suggest a suitable possible method to fix each of these issues** and **3)provide your justification for using your suggested fix method.** Use the table below to organise your findings and analysis, and add more rows if needed

[4 Marks]

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	age	Extreme/Impossible values	Used a filter to return appropriate values	The retained age variable had values like age -30 and age 180, the fixing introduced reliable age data
2	employment_length	Extreme/Impossible values	Used a filter to return appropriate values	The variable had extreme values of 123 and 130 years of employment, impossible values that had to be dropped
3	loan_interest_rate	Extreme/Impossible values	Used a filter to return appropriate values	This variable had values like -11.14 and 200 which are extremely impossible values for a bank to provide.
4	max_loan_amount(allowed)	Extreme/Impossible values	Used a filter to return appropriate values	This variable had minus values as well as value 12467989660, which is an impossible value for a bank to provide
5	age	Data missingness	Imputed data for the missing values by calculating mean of the dataset	The dataset cannot have missing data, the best option was to impute the data as the missingness was low
6	income	Data missingness	Imputed data for the missing values by calculating mean of the dataset	The dataset cannot have missing data, the best option was to impute the data as the missingness was low
7	employment_length	Data missingness	Imputed data for the missing values by calculating mean of the dataset	The dataset cannot have missing data, the best option was to impute the data as the missingness was low
8	loan_interest_rate	Data missingness	Imputed data for the missing values by calculating mean of the dataset	The dataset cannot have missing data, the best option was to impute the data as the missingness was low
9	loan_approval_status	Irregular data values	Mapped the irregular data to numeric values	Data had instances that meant related in meaning but not in labels, this label encoding fix ensures that data can easily find relations
10	payment_default_on_file	Irregular data values	Mapped the irregular data to numeric values	Data had instances that meant related in meaning but not in labels, this label encoding fix ensures that data can easily find relations
11	<i>All Remaining</i>	Dropped the remaining missing variables	Dropped the remaining missing data	The remaining data was dropped, After numeric encoding many indexes have a NaN variable, this is not useful and is dropped by dropna()

:	:	:	:	:
---	---	---	---	---

Task (3) – Data Preparation: Cleaning and Transforming your data

b) With the aid of Python packages and your Final Python Notebook 1, implement your suggested fixes of issues in the previous Task 3-a in your final Python Notebook1.

1) Show evidence (before and after) of implementing your suggested fix to the problems you identified for your dataset in Task 3-a.

To show your evidence, paste screenshots of your relevant code OUTPUTS ONLY (Do not paste the code).

2) Indicate and annotate the issue and the fix in each of your provided evidence screenshots.

[4 Marks]

Output screenshot of the issue before the fix	Output screenshot after fixing the issue
---	--

Before

1 – 4

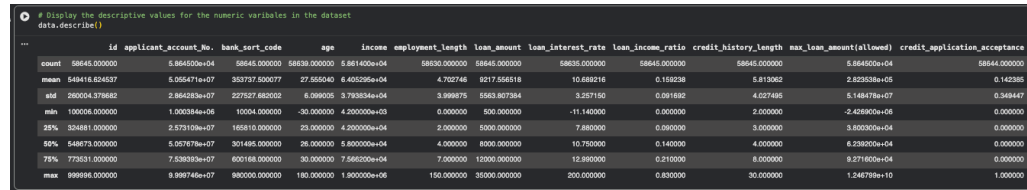


Figure 4 - Age, employment_length, loan_interest_rate, max_loan_amount(allowed) before value outlier removal

Fix

1 – 4



Figure 5 - Filter Age, employment_length, loan_interest_rate, max_loan_amount(allowed)

After

1 - 4

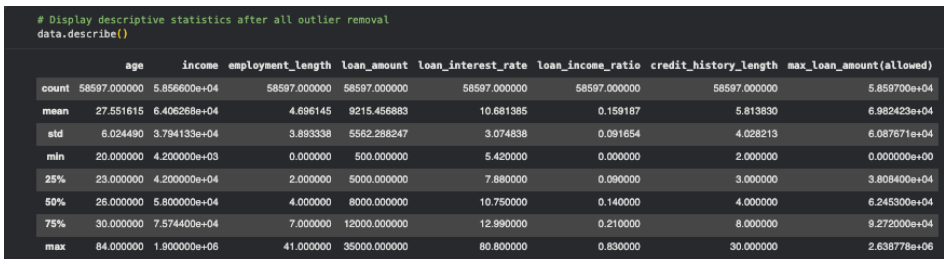


Figure 6 - Age, employment_length, loan_interest_rate, max_loan_amount(allowed) after value outlier removal

Before

5 – 8

Return a count of the rows with null variables as numbers and percentages (Code Reuse Session 1 (Week 3) snippet 10,11)

```
# Counting and displaying the number of missing values per variable
data.isnull().sum()
```

```
0
id 0
applicant_account_No. 0
bank_sort_code 0
age 6
sex 58283
education_qualifications 0
income 31
home_ownership 0
employment_length 15
loan_intent 0
loan_amount 0
loan_interest_rate 10
loan_income_ratio 0
payment_default_on_file 5
credit_history_length 0
loan_approval_status 1
max_loan_amount(allowed) 0
credit_application_acceptance 1
```

dtype: int64

Figure 7 - Age, income, employment_length, loan_interest_rate before data imputation

Fix

5 - 8

```
# Calculate the mean of the age variable
mean_age = data['age'].mean()
# Fill missing age values using the mean age value
data['age'].fillna(mean_age, inplace=True)

~/ipykernel_12579/451621765.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col, value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

# Calculate the mean of the income variable
mean_income = data['income'].mean()
# Fill missing income values using the mean income value
data['income'].fillna(mean_income, inplace=True)

~/ipykernel_12579/4854745385.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col, value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

# Calculate the mean of the employment_length variable
mean_employment_length = data['employment_length'].mean()
# Fill missing employment_length values using the mean employment_length value
data['employment_length'].fillna(mean_employment_length, inplace=True)

~/ipykernel_12579/263698916.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col, value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.

# Calculate the mean of the loan_interest_rate variable
mean_loan_interest_rate = data['loan_interest_rate'].mean()
# Fill missing loan_interest_rate values using the mean loan_interest_rate value
data['loan_interest_rate'].fillna(mean_loan_interest_rate, inplace=True)

~/ipykernel_12579/118384628.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col, value, inplace=True)' or 'df[col] = df[col].method(value)' instead, to perform the operation inplace on the original object.
```

Figure 8 - Age, income, employment_length, loan_interest_rate imputation fix

After

5 - 8

```
# Confirm the remaining number of missing values per variable after mean imputation
data.isnull().sum()
```

```
0
age 0
income 0
home_ownership 0
employment_length 0
loan_intent 0
loan_amount 0
loan_interest_rate 0
loan_income_ratio 0
payment_default_on_file 0
credit_history_length 0
loan_approval_status 0
max_loan_amount(allowed) 0
```

dtype: int64

Figure 9 - Age, income, employment_length, loan_interest_rate after imputation fix

Before

9 - 10

```

# # Display the unique values in the loan_approval_status varibale
data['loan_approval_status'].unique()

array(['Approved', 'APPROVED', 'APPROVED ', 'Declined ', 'DECLINED ',
       'Reject', nan, 'Accept', 'Accept '], dtype=object)

# # Display the unique values in the payment_default_on_file varibale
data['payment_default_on_file'].unique()

array(['N', 'Y', nan, 'NO', 'YES'], dtype=object)

```

Figure 10 - loan_approval_status, payment_default_on_file before categorical fix

Fix

9 – 10

```

Formatting categorical values into numeric (Code Reuse Session 1 (Week 3) snippet 23)

# Mapping the loan_approval_status labels into numeric values
data['loan_approval_status'] = data['loan_approval_status'].map({
    'Approved': 1, 'APPROVED': 1, 'APPROVED ': 1, 'Accept ': 1, 'Accept': 1,
    'Declined ': 0, 'Declined': 0, 'Reject': 0, 'DECLINED ': 0, 'DECLINED': 0
})

# Mapping the payment_default_on_file labels into numeric values
data['payment_default_on_file'] = data['payment_default_on_file'].map({
    'N':0, 'NO':0,
    'Y':1, 'YES':1
})

```

Figure 11 - loan_approval_status, payment_default_on_file categorical fix

After

9 - 10

```

# Display the distinct values of payment_default_on_file after mapping
data['payment_default_on_file'].unique()

array([ 0.,  1., nan])

# Display the distinct values of loan_approval_status after mapping
data['loan_approval_status'].unique()

array([ 1.,  0., nan])

```

Figure 12 - loan_approval_status, payment_default_on_file after categorical fix

Before

11

```

# Confirm the remaining number of missing values per variable
data.isnull().sum()

...
age      0
income   0
home_ownership  0
employment_length  0
loan_intent  0
loan_amount  0
loan_interest_rate  0
loan_income_ratio  0
payment_default_on_file  5
credit_history_length  0
loan_approval_status  1
max_loan_amount(allowed)  0

dtype: int64

```

Figure 13 - payment_default_on_file, loan_approval_status before removing NaN values

Fix

11

```

# Drop the missing valued rows
data = data.dropna()

```

Figure 14 - payment_default_on_file, loan_approval_status removing NaN values

After
11

```
# Confirm the remaining number of missing values per variable
data.isnull().sum()

0
age      0
Income   0
home_ownership  0
employment_length  0
loan_intent  0
loan_amount  0
loan_interest_rate  0
loan_income_ratio  0
payment_default_on_file  0
credit_history_length  0
loan_approval_status  0
max_loan_amount(allowed)  0
```

Figure 15 - payment_default_on_file, loan_approval_status after removing NaN values

Output screenshot of the issue before the fix	Output screenshot after fixing the issue
---	--

Task (4) – Classification Modelling of Clients Loan Approval Status

a) In your Final Python Notebook 2, you built THREE different models to predict loan approval status: Logistic Regression (LR), K Nearest Neighbour (KNN) and Naïve Bayes (NB). These algorithms are a mix of parametric and non-parametric algorithms.

1) Note down the type of each algorithm (parametric vs non-parametric),

2) name any learnable parameters, and

3) list any strategic hyperparameters for each algorithm which you want to consider tuning. Organise your answer in the table below:

[3 Marks]

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Parametric	nb.class_prior – the prior probabilities nb.thea_ - the mean of each fearure nb.var_ - the variance of each feature	priors
Logistic Regression (LR)	Parametric	logreg.intercept_ The intercept (bias) logreg.coef_ The coeffiecients (weights)	max_iter
K-Nearest Neighbour (KNN)	Non-parametric	None as KNN is a lazy learner	n_neighbors

Task (4) – Classification Modelling of Clients Loan Approval Status

b) With the aid of your Final Python Notebook 2, use the training–test split approach with your retained applicable input features only and the target output feature to build your predictive classification models.

[3 Marks]

i. Screenshot

- 1) the list of all feature names used for building your classification models and the corresponding
- 2) data shape function output. (Paste screenshots of the relevant code output only; do not paste the Python code).

```
# Declare the list of input features for modeling
feature_cols = ['age', 'income', 'home_ownership', 'employment_length',
               'loan_intent', 'loan_amount', 'loan_interest_rate',
               'loan_income_ratio', 'payment_default_on_file', 'credit_history_length']

# Assign all input features to x
X = data[feature_cols] # Features

# Assign the target variable of 'loan_approval_status' to y
y = data['loan_approval_status']

# Return the list of all the features
feature_cols

['age',
 'income',
 'home_ownership',
 'employment_length',
 'loan_intent',
 'loan_amount',
 'loan_interest_rate',
 'loan_income_ratio',
 'payment_default_on_file',
 'credit_history_length']
```

Figure 16 - List of all the feature names for `feature_cols`

```
# Check that the split was applied correctly
X_train.shape

(46872, 10)

# Check that the split was applied correctly
X_test.shape

... (11719, 10)
```

Figure 17 - The dimensions of the dataset

- ii. In less than 150 words, research and justify (defend) your choice of the training-test split ratio and provide an in-text citation.

The training test split of 80/20 was chosen for this task, the main factor of its popularity however in the literature by (Vrigazova,2021), it was found that an 80,20 split could still preserve accuracy for classification methods especially for decision tress.

As stated the 80/20 is seen as common practises in machine learning research (Sivakumar, 2024) and it offers a good balance by offering enough data for both training and testing.

References

Vrigazova, B. (2021) 'The proportion for splitting data into training and test set for the bootstrap in classification problems', *Business Systems Research*, 12(1), pp. 228–242. doi:10.2478/bsrj-2021-0015.

Sivakumar, M., Parthasarathy, S., & Padmapriya, T. (2024). Trade-off between training and testing ratio in machine learning for medical image processing. *PeerJ. Computer science*, 10, e2245. <https://doi.org/10.7717/peerj-cs.2245>

- iii. Provide as evidence the code block line and code output from your Final Python Notebook 2 that ensures two conditions:

- 1) all your models were tested on the same test instances (clients) in your dataset;
- 2) the labels ratio of approval Status “Accept” to “Reject” is the same in the training and test subsets.
- 3) State the training-test split function parameters in your code line that are responsible for meeting both conditions.
- 4) In less than 150 words, research and justify (defend) your decision to implement both conditions in your Python Notebook 2 notebook with in-text citations where possible.

Using the imported library now we assign parameters to the function for training and testing (Code Reuse Session 2 (Week 5) snippet 8)

```
1)
0s
# First assign x and y to the training and testing subsets, followed by allocating testing data to 20% leaving training data as 80%
# The random_state ensure that when running this code multiple times the instance for testing and training do not change
# Finally stratify ensure that the ratio of approved and declined is the same in both data sets
X_train,X_test,y_train,y_test = train_test_split(X , y, test_size=0.2, random_state=42, stratify=y)
```

Figure 18 - Proof showing all models are on the same test instances

Both conditions are implemented across the 3 classification models, firstly stratify ensures that the parameters of the approved and declined classes are kept and preserved in their original training and test sets, the data split will be generated consistently for the models. Next was the fixed random seed (random_state= 42), which was executed to ensure that when testing is ongoing the dataset stays split the same throughout.

References with in-text citation

Task (5) – Evaluating your Loan Approval Status Classification Models

Your finance team provided the following success criteria to guide you when evaluating and selecting your best model: *“When evaluating your model's performance, which addresses your first research question (a). The model is expected to misclassify clients' applications. Thus, the model should aim to predict as many “Reject” status for clients as many as possible to decrease the risk of future defaulted loan payments. However, the model should demonstrate that its high “Reject” prediction rate is mainly due to a larger number of correctly detected (predicted) rejected loan applications.”*

a) With the aid of Final Python Notebook 2, for each of your models (Logistic Regression LR, Naive Bayes NB and K-Nearest Neighbours KNN)

- 1) paste the test confusion matrix,
- 2) the classification report and
- 3) the AUC-ROC curve graphs

as screenshots from the output of your Python code.

[3 Marks]

1) Screenshot of the test confusion matrix for (NB, LR, and KNN); make sure you title each matrix with its algorithm name

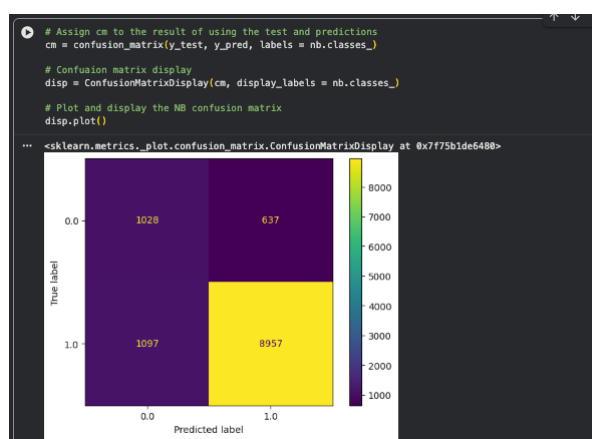


Figure 19 - NB confusion matrix

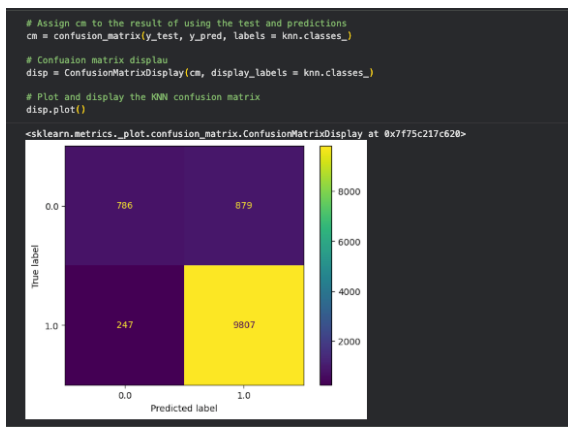


Figure 20 - KNN confusion matrix

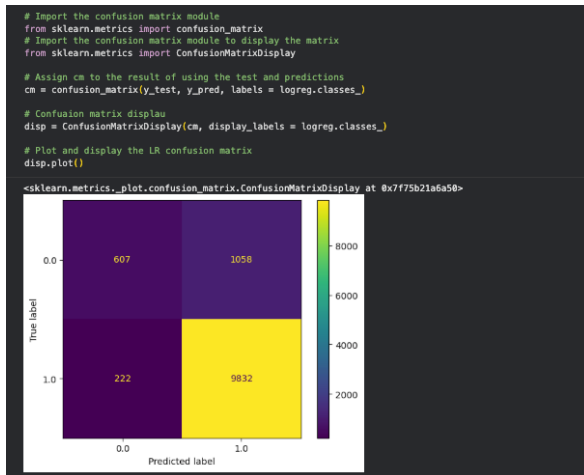


Figure 21 - LR confusion matrix

2) Screenshot of the classification report for (NB, LR, and KNN); make sure you title each classification report with its algorithm name

```

# Import the classification_report to produce precision, recall, F1 score and support
from sklearn.metrics import classification_report
# Print the full classification report after import
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0.0	0.73	0.36	0.49	1665
1.0	0.90	0.98	0.94	10054
accuracy			0.89	11719
macro avg	0.82	0.67	0.71	11719
weighted avg	0.88	0.89	0.87	11719

Figure 22 - LR classification report

```

# Print the full classification report for the KNN model
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0.0	0.76	0.47	0.58	1665
1.0	0.92	0.98	0.95	10054
accuracy			0.90	11719
macro avg	0.84	0.72	0.76	11719
weighted avg	0.90	0.90	0.89	11719

Figure 23 - KNN classification report

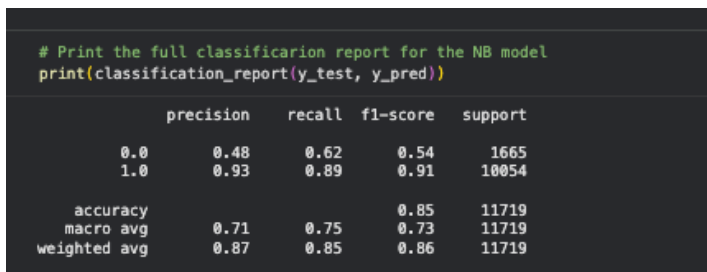


Figure 24 - NB classification report

3) Screenshot of the AUC-ROC Curve for (NB, LR, and KNN); make sure you title each graph with its algorithm name

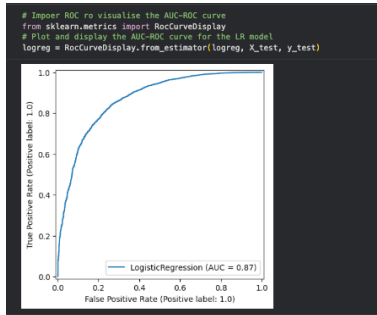


Figure 25 - LR AUC

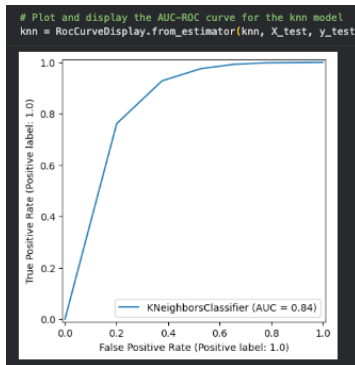


Figure 26 - KNN ROC

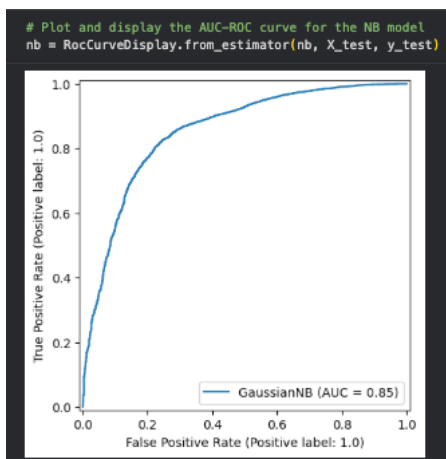


Figure 27 - NB ROC

Task (5) – Evaluating your Loan Approval Status Classification Models

b) Five different classification evaluation metrics are calculated in your [Final Python Notebook 2](#).

1) State which evaluation metric/metrics to “USE or “DO NOT USE” to closely interpret the above success criteria.

2) For justification, explain how closely your choice of “USE” or “DO NOT USE” for a metric interprets the given success criteria.

With the aid of your [Final Python Notebook 2](#),

3) document all the TEST SCORES for each built model in the table below.

[7 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing "USING" or "NOT USING" this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	DON'T USE	Accuracy is extremely misleading, based on its definition it would assume correctness however model would fail on imbalanced dataset	NB	0.85
			LR	0.89
			KNN (k=5)	0.9
Recall	USE	Recall measure the actual declines applications the model identifies, this is the success criteria	NB	0.62
			LR	0.36
			KNN (k=5)	0.47
Precision	USE	The precision measure the declined predictions that are actually declined, this address the criteria	NB	0.49
			LR	0.73
			KNN (k=5)	0.76
F1-score	USE	F1 score is the mean of precision and recall and it offers the best data based on the success criteria	NB	0.54
			LR	0.49
			KNN (k=5)	0.58
AUC-Roc	DON'T USE	AUC measures how a model differentiates between class at different thresholds, it doesn't really adhere to the success criteria	NB	0.85
			LR	0.87
			KNN (k=5)	0.84

Task (5) – Evaluating your Loan Approval Status Classification Models

c) **Suggest a single best approval status classification model based on the 'USED' performance metrics scores you identified in Task (5-b). In less than 100 words, briefly describe how well your best model satisfies the needs of your finance team.** **[2 Marks]**

Based on the USED performance metrics the best model to use for predicting loan approval status was KNN with K = 5. KNN achieves the highest F1 score for the Declined class which means there is a strong balance between the recall and precision. It also does adhere to the success criteria by catching a meaningful proportion of declined applications, which in terms of finance team's requirements is fulfilled.

Task (5) – Evaluating your Loan Approval Status Classification Models

d) To enhance your selected best model/s performance (from Task 5-c), tune some of its possible hyperparameters, which you indicated in Task (4-a) for that specific algorithm. With the aid of Final Python Notebook 2, **Re-train and test the best algorithm again with GridSearchCV** **[5 Marks]**

i. With the aid of your Final Python Notebook 2,

1) Paste into this report the line of code which shows evidence of specifying a parameters grid and applying the GridSearchCV function to rebuild your selected best model.

2) Then, document the estimated best hyperparameters for the optimised model.

Based on the highest F1 score for class 0.0 at 0.58 the best chosen model will be KNN, in which case it will be tuned to find the best hyperparameter combination (Code Reuse Session 2 (Week 5) snippet 25)

```
# KNN is reinstantiated due to the fact that it was previously assigned to ROC,
# it will be make empty to begin the tuning process
knn = KNeighborsClassifier()
# Creating a dictionary for the hyperparameters to search through, n_neighbors controls how many points the
# algorithm consider during prediction
param_grid = {'n_neighbors': [1, 3, 5, 7, 9, 11, 13, 15]}
# instantiate grid search to knn and take the arguments of knn for tuning, param_grid for testing and
# 5 fold cross validation for inner training
knn_gscv = GridSearchCV(knn, param_grid, cv = 5)
# Where GridSearchCV tries every number of n_neighbors to find the best combination
knn_gscv.fit(X_train,y_train)
# Displays the best combination
knn_gscv.best_params_

{'n_neighbors': 13}
```

Figure 28 - Evidence of GridSearchCV

Best Hyperparameters = {'n_neighbors': 13}

ii. With the aid of your Final Python Notebook 2,

- 1) paste into this report the test confusion matrix for your best model before and after hyperparameter tuning.
- 2) Also, document the new score/s of the “USED” performance metric/s of your choice to interpret the success criteria indicated in Task (5.b) before and after tuning.
- 3) Comment on whether the tuning of hyperparameters of your best model improved its positive predictive ability in line with the success criteria.

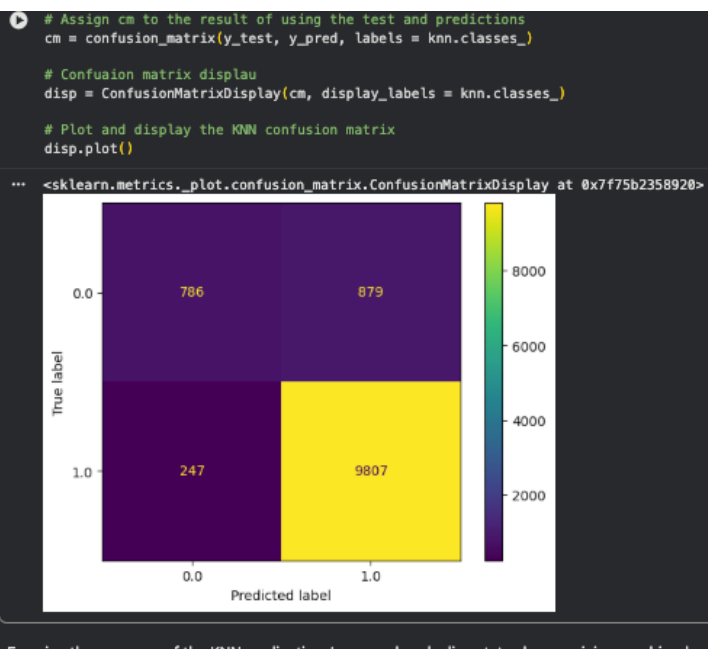


Figure 29 - KNN before tuning (KNN = 5)

```
# Print the full classification report for the tuned KNN model
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0.0	0.82	0.44	0.57	1665
1.0	0.91	0.98	0.95	10054
accuracy			0.91	11719
macro avg	0.87	0.71	0.76	11719
weighted avg	0.90	0.91	0.89	11719

Figure 30 - KNN after tuning

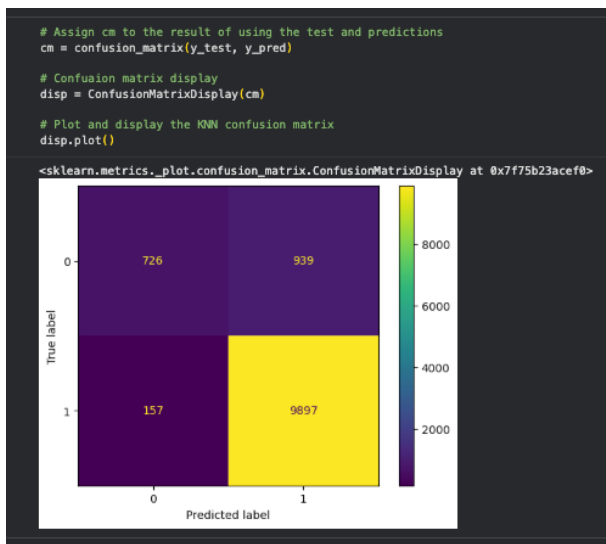


Figure 31 - KNN after tuning (KNN = 13)

```

# Print the full classification report for the tuned KNN model
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0.0	0.82	0.44	0.57	1665
1.0	0.91	0.98	0.95	10854
accuracy			0.91	11719
macro avg	0.87	0.71	0.76	11719
weighted avg	0.90	0.91	0.89	11719

Figure 32 - KNN after tuning

K=13 as the optimal number of neighbour, tuning has improved greatly as can be seen in the precision in the declined class which increased to 0.82 which now means there is a larger chance of declined status being correct, this is inline with the success criteria set by the finance team.